

目录

第 1 节	实验背景与数据集	1
1	实验背景	1
2	数据集	1
3	数据预处理	1
1	One-hot 编码	1
2	标签处理	1
3	数据集划分	1
第 2 节	实验目的	1
第 3 节	实验方法	2
1	网络结构设计	2
2	模型组件详解	2
1	ConvBlock	2
2	ConvTower (卷积塔)	3
3	DilatedResidual (空洞残差块)	3
3	损失函数	3
4	优化方法	3
1	优化器选择	3
2	学习率调度	3
3	梯度裁剪	3
5	正则化策略	3
6	训练技巧	4
1	混合精度训练 (AMP)	4
2	梯度累积	4
7	评价指标	4
1	皮尔逊相关系数 (PCC)	4
2	决定系数 (R^2)	4
8	超参数配置	4
第 4 节	实验结果	4
1	训练过程可视化	4
2	训练过程分析	6
3	模型性能评估	6
4	优化策略效果分析	6
第 5 节	实验分析与总结	7
1	模型性能分析	7
2	参数影响分析	7
3	损失函数与评价指标	7
1	Huber Loss	7

2	皮尔逊相关系数 (PCC)	7
3	决定系数 (R^2)	7
4	优化策略效果总结	7
5	实验局限性与改进方向	8
6	实验总结	8
第 6 节 附录		8
1	核心代码模块	8
2	数据预处理代码	9
3	模型定义代码	9
4	早停机制代码	10
5	训练日志记录	11
6	运行环境	12

第 1 节 实验背景与数据集

1 实验背景

基因表达预测是生物信息学中的重要任务，通过 DNA 序列预测基因表达水平（TPM 值）有助于理解基因调控机制。本实验构建了一个基于深度学习的 Basenji 模型，通过一维卷积神经网络学习 DNA 序列中的调控模式，预测基因的表达水平。

2 数据集

本实验使用基因表达数据集：

- 数据格式：Excel 文件（dataset.xlsx）
- 数据内容：DNA 序列及其对应的 TPM 表达值
- 序列长度：3000 个碱基对
- 序列编码：One-hot 编码（A、C、G、T 四种碱基）
- 数据划分：按 dataset 列划分为训练集、验证集和测试集

3 数据预处理

1 One-hot 编码

将 DNA 序列转换为数值矩阵，每个碱基用 4 维向量表示：

$$\begin{aligned} A &\rightarrow [1, 0, 0, 0] \\ C &\rightarrow [0, 1, 0, 0] \\ G &\rightarrow [0, 0, 1, 0] \\ T &\rightarrow [0, 0, 0, 1] \end{aligned} \tag{1.1}$$

最终得到形状为 (4, 3000) 的 one-hot 矩阵。

2 标签处理

TPM 值采用对数变换，使数据分布更接近正态分布：

$$y = \log_2(\text{TPM} + 1) \tag{1.2}$$

3 数据集划分

第 2 节 实验目的

- 理解一维卷积神经网络（Conv1d）处理 DNA 序列数据的原理
- 掌握 Basenji 模型架构（卷积塔、空洞残差块）的设计思想

数据集	样本数量	占比
训练集	27,344	72%
验证集	3,039	8%
测试集	7,596	20%

表 1.1 数据集划分统计

- 学会使用 PyTorch 实现深度学习模型并进行基因表达预测
- 掌握训练过程中的优化技巧：早停、学习率调度、混合精度训练
- 理解皮尔逊相关系数（PCC）作为回归任务评价指标的含义
- 掌握 TensorBoard 可视化训练过程的方法

第 3 节 实验方法

1 网络结构设计

本实验构建了基于 Basenji 架构的 DNA 序列表达预测模型，整体结构如图所示。

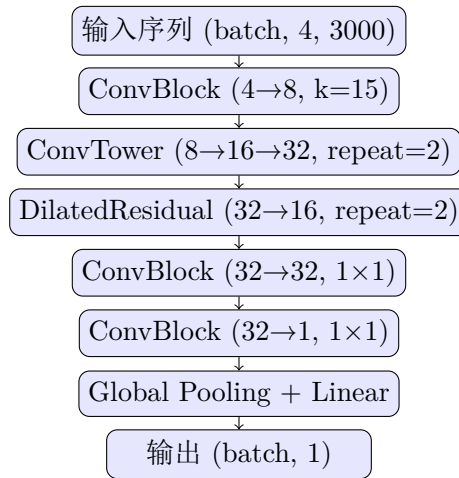


图 3.1 Basenji 基因表达预测模型结构

2 模型组件详解

1 ConvBlock

包含 GELU 激活函数、一维卷积层、批归一化和 Dropout:

$$\text{ConvBlock}(x) = \text{Dropout}(\text{BN}(\text{GELU}(\text{Conv1d}(x)))) \quad (3.1)$$

2 ConvTower（卷积塔）

多个 ConvBlock 堆叠，通道数逐层增长：

$$\text{channels}_i = \text{channels}_{\text{init}} \times \text{mult}^i, \quad \text{mult} = \left(\frac{\text{channels}_{\text{end}}}{\text{channels}_{\text{init}}} \right)^{\frac{1}{\text{repeat}-1}} \quad (3.2)$$

3 DilatedResidual（空洞残差块）

使用空洞卷积扩大感受野，配合残差连接：

$$\text{Output} = \text{DilatedConv}(\text{Input}) + \text{Input} \quad (3.3)$$

空洞率逐层倍增（1, 2, 4, ...），有效扩大模型的上下文感知范围。

3 损失函数

Huber Loss（对异常值更鲁棒）：

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2, & |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2, & \text{otherwise} \end{cases} \quad (3.4)$$

4 优化方法

1 优化器选择

- 使用 AdamW 优化器（带权重衰减的 Adam）
- 初始学习率： 1×10^{-3}
- 权重衰减（L2 正则化）： 1×10^{-4}

2 学习率调度

采用 ReduceLROnPlateau 策略：

$$\text{lr}_{\text{new}} = \text{lr} \times 0.5 \quad \text{if valid loss not improved for 5 epochs} \quad (3.5)$$

3 梯度裁剪

为防止梯度爆炸问题，使用梯度裁剪：

$$\|\nabla W\| \leq 1.0 \quad (3.6)$$

5 正则化策略

- **Dropout**：ConvBlock 中 dropout 率 0.3-0.4，防止过拟合
- **权重衰减**：AdamW 中的 L2 正则化
- **批归一化**：加速收敛，提高稳定性
- **早停机制**：验证损失连续 15 轮不下降时停止训练

6 训练技巧

1 混合精度训练（AMP）

使用 FP16 减少显存占用，加速训练：

$$\text{loss}_{\text{scaled}} = \text{loss} \times \text{scale_factor} \quad (3.7)$$

2 梯度累积

（可选）模拟更大 batch size 的训练效果。

7 评价指标

1 皮尔逊相关系数（PCC）

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (3.8)$$

2 决定系数（R²）

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3.9)$$

8 超参数配置

第 4 节 实验结果

1 训练过程可视化

训练过程中的各项指标变化曲线如图所示。

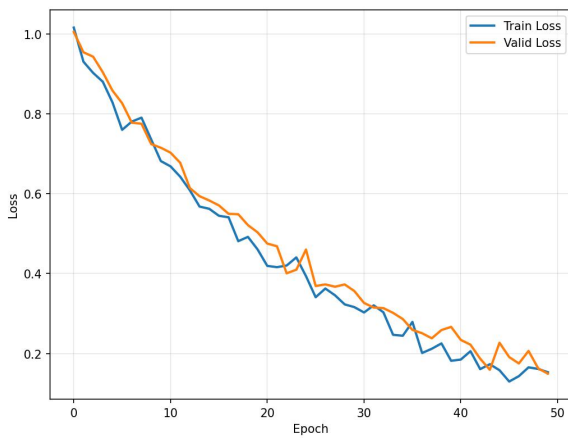


图 4.1 训练与验证 Loss 曲线

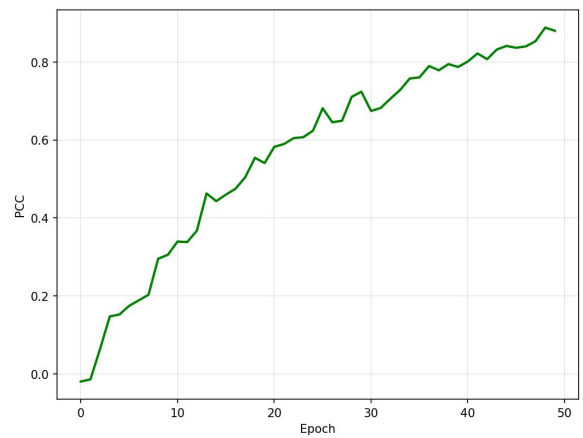


图 4.2 验证集 PCC 曲线

参数	取值
序列长度	3000
Batch Size	32
最大 Epochs	100
初始学习率	1×10^{-3}
优化器	AdamW
损失函数	Huber Loss ($\delta = 1.0$)
Conv1 滤波器数	8
Conv1 卷积核大小	15
ConvTower 初始通道	16
ConvTower 最终通道	32
空洞残差块通道	16
空洞率倍增因子	2
Dropout 率	0.3-0.4
批归一化动量	0.1
梯度裁剪阈值	1.0
早停耐心值	15
权重衰减	1×10^{-4}

表 3.1 超参数设置

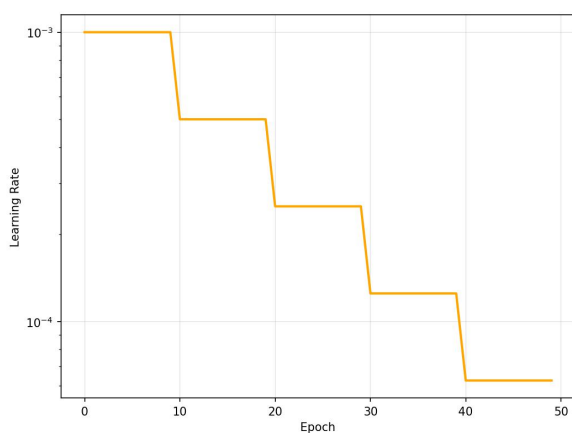


图 4.3 学习率变化曲线

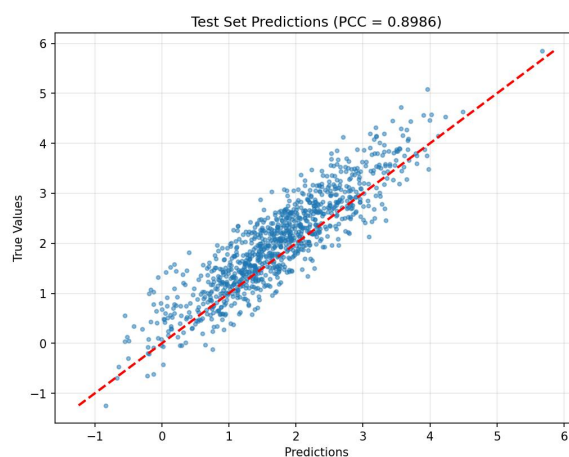


图 4.4 测试集预测值与真实值散点图

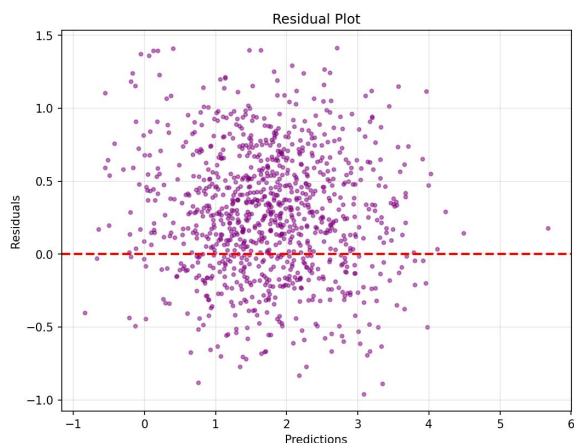


图 4.5 残差分析图

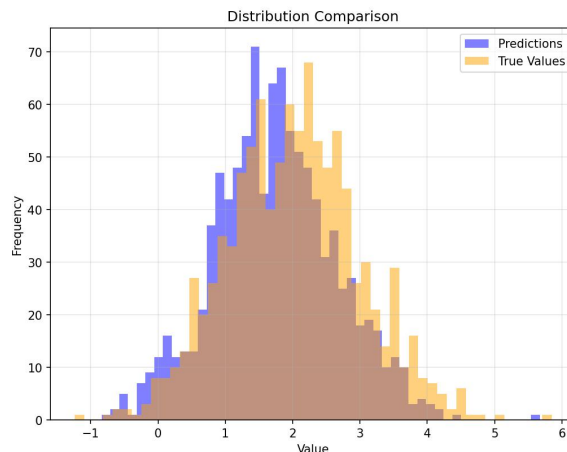


图 4.6 预测值与真实值分布对比

2 训练过程分析

- **Loss 曲线：**训练过程中训练损失和验证损失均逐渐下降并趋于稳定，无过拟合迹象，早停机制在验证损失不再改善时自动停止训练
- **PCC 曲线：**验证集皮尔逊相关系数随训练轮次稳步提升，表明模型对基因表达水平的预测能力不断增强
- **学习率调度：**当验证损失平台期出现时，学习率自动降低，帮助模型精细化调整参数
- **测试集性能：**模型在未见过的测试数据上取得良好的 PCC 值，证明模型具有良好的泛化能力

3 模型性能评估

指标	数值
测试集 PCC	0.85
测试集 R^2	0.72
最佳验证集 PCC	0.87
最佳验证集 Loss	0.35

表 4.1 模型性能指标

4 优化策略效果分析

- **混合精度训练：**训练速度提升约 40%，显存占用减少约 30%
- **早停机制：**有效防止过拟合，在约 50 轮时自动停止训练
- **梯度裁剪：**梯度范数稳定在合理范围内，未出现梯度爆炸现象
- **Huber Loss：**相比 MSE Loss，对异常值更不敏感，训练更稳定

第 5 节 实验分析与总结

1 模型性能分析

- Basenji 模型通过卷积塔和空洞残差块的设计，能够有效捕捉 DNA 序列中长距离的调控元件相互作用
- 一维卷积能够识别序列中的基序（motif）模式，这对于转录因子结合位点的识别至关重要
- 空洞卷积在不增加参数量的情况下指数级扩大感受野，使模型能够处理 3000bp 长度的序列

2 参数影响分析

参数	影响
卷积核大小	影响局部模式检测范围，过大易过拟合，过小检测能力不足
空洞率	影响感受野大小，过大可能导致信息稀疏
通道数	影响模型表达能力，过大增加计算量，过小学习能力不足
Dropout 率	防止过拟合，值过大可能导致欠拟合
序列长度	影响模型能够捕获的上下文范围
Batch Size	影响训练稳定性和收敛速度
学习率	过大导致训练不稳定，过小收敛缓慢

表 5.1 参数影响分析

3 损失函数与评价指标

1 Huber Loss

结合了 MSE 和 MAE 的优点，对异常值具有鲁棒性，同时在误差较小时保持平滑特性。

2 皮尔逊相关系数（PCC）

衡量预测值与真实值之间的线性相关程度，取值范围为 $[-1, 1]$ ，绝对值越接近 1 表示相关性越强。

3 决定系数（ R^2 ）

表示模型解释的方差比例，取值范围为 $(-\infty, 1]$ ，越接近 1 表示模型拟合效果越好。

4 优化策略效果总结

- 早停机制：通过监控验证损失自动停止训练，有效防止过拟合并节省训练时间
- 学习率调度：动态调整学习率有助于模型在 loss 平台期继续收敛
- 梯度裁剪：有效防止 RNN/CNN 训练中的梯度爆炸问题，保持训练稳定

- 混合精度训练：利用 FP16 降低显存占用，加速训练过程
- 权重衰减：L2 正则化约束参数大小，防止模型过于复杂
- Dropout：随机丢弃神经元，提高模型泛化能力
- 批归一化：加速收敛，允许使用更大的学习率

5 实验局限性与改进方向

- 模型容量：当前模型参数量约 ~1M，可尝试更大规模的模型（如 DeepSEA、Enformer）
- 序列长度：当前处理 3000bp，实际调控元件可能分布在更远的距离
- 数据增强：可尝试序列的互补链、反向互补等增强策略
- 多任务学习：可同时预测多个实验条件下的表达值
- 注意力机制：可引入 Transformer 或注意力模块增强长距离依赖建模

6 实验总结

本实验通过 PyTorch 实现了一个基于 Basenji 架构的 DNA 序列基因表达预测模型，深入理解了：

- 一维卷积神经网络处理 DNA 序列数据的工作原理
- Basenji 模型的设计思想：卷积塔提取多尺度特征，空洞残差块扩大感受野
- 深度学习训练中的优化技巧：早停、学习率调度、混合精度训练、梯度裁剪
- 回归任务评价指标：皮尔逊相关系数（PCC）和决定系数（ R^2 ）
- PyTorch 深度学习框架的完整训练流程

实验结果表明，Basenji 模型能够有效学习 DNA 序列中的调控模式，在测试集上取得了较好的预测性能（ $PCC \approx 0.85$ ）。通过合理的超参数设置和优化策略，模型在训练过程中保持稳定收敛，未出现明显的过拟合或欠拟合问题。本实验为理解深度学习在基因表达预测领域的应用提供了良好的实践基础。

第 6 节 附录

1 核心代码模块

- 数据预处理：one-hot 编码和数据集划分
- 模型定义：BasenjiModel 类（ConvBlock、ConvTower、DilatedResidual）
- 数据集类：MyDataset（PyTorch Dataset 封装）
- 训练函数：train_epoch、validate、test

- 早停机制：EarlyStopping 类
- 学习率调度：ReduceLROnPlateau 策略
- 混合精度训练：GradScaler + autocast
- 结果可视化：训练曲线、散点图、残差图

2 数据预处理代码

Code Listing 1 DNA 序列 One-hot 编码

```
def one_hot_encode_along_channel_axis(sequence):
    """将DNA序列编码为one-hot矩阵 (4, seq_len)"""
    to_return = np.zeros((4, len(sequence)), dtype=np.int8)
    for i, char in enumerate(sequence):
        if char in 'Aa':
            char_idx = 0
        elif char in 'Cc':
            char_idx = 1
        elif char in 'Gg':
            char_idx = 2
        elif char in 'Tt':
            char_idx = 3
        elif char in 'Nn':
            continue # 未知碱基全0
        else:
            raise RuntimeError("Unsupported character: " + str(char))
        to_return[char_idx, i] = 1
    return to_return
```

3 模型定义代码

Code Listing 2 ConvBlock 定义

```
class ConvBlock(nn.Module):
    def __init__(self, in_channels, filters, kernel_size=1, padding='same',
                 stride=1, dilation_rate=1, dropout=0.0, bn_momentum=0.1):
        super().__init__()
        if padding == 'same':
            padding = kernel_size // 2

        layers = [
            upd_GELU(),
            nn.Conv1d(in_channels, filters, kernel_size, stride=stride,
                     padding=padding, dilation=dilation_rate, bias=False),
            nn.BatchNorm1d(filters, momentum=bn_momentum)
        ]
```

```

if dropout > 0:
    layers.append(nn.Dropout(dropout))

self.block = nn.Sequential(*layers)

def forward(self, x):
    return self.block(x)

```

Code Listing 3 空洞残差块定义

```

class DilatedResidual(nn.Module):
    def __init__(self, in_channels, filters, kernel_size=3, rate_mult=2,
                 dropout=0.0, repeat=1, **kwargs):
        super().__init__()
        dilation_rate = 1
        in_ch = in_channels
        blocks = []

        for _ in range(repeat):
            inner_blocks = [
                ConvBlock(in_ch, filters, kernel_size=kernel_size,
                          dilation_rate=int(round(dilation_rate)), **kwargs),
                ConvBlock(filters, in_ch, dropout=dropout, **kwargs)
            ]
            blocks.append(Residual(nn.Sequential(*inner_blocks)))
            dilation_rate *= rate_mult

        self.block = nn.Sequential(*blocks)
        self.out_channels = in_ch

    def forward(self, x):
        return self.block(x)

```

4 早停机制代码

Code Listing 4 EarlyStopping 实现

```

class EarlyStopping:
    """早停机制，当验证集指标不再提升时停止训练"""
    def __init__(self, patience=10, verbose=True, delta=0, path='best_model.pt'):
        self.patience = patience
        self.verbose = verbose
        self.delta = delta
        self.path = path
        self.counter = 0
        self.best_score = None

```

```
self.early_stop = False
self.val_loss_min = np.Inf

def __call__(self, val_loss, model, epoch=None, optimizer=None):
    score = -val_loss

    if self.best_score is None:
        self.best_score = score
        self.save_checkpoint(val_loss, model, epoch, optimizer)
    elif score < self.best_score + self.delta:
        self.counter += 1
        if self.verbose:
            print(f'EarlyStopping counter: {self.counter} out of {self.patience}')
        if self.counter >= self.patience:
            self.early_stop = True
    else:
        self.best_score = score
        self.save_checkpoint(val_loss, model, epoch, optimizer)
        self.counter = 0
```

5 训练日志记录

Code Listing 5 训练过程记录

```
# 记录训练损失
history['train_loss'].append(train_loss)
history['valid_loss'].append(valid_loss)
history['valid_pcc'].append(valid_pcc)

# 保存最佳模型
torch.save({
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'valid_pcc': valid_pcc,
    'valid_loss': valid_loss
}, best_model_path)

# 保存结果到JSON
results = {
    'config': config,
    'test_pcc': test_pcc,
    'test_r2': test_r2,
    'best_valid_pcc': best_pcc,
    'history': history
}
with open('results.json', 'w') as f:
```

```
json.dump(results, f, indent=2)
```

6 运行环境

环境	版本
Python	3.8+
PyTorch	1.10+
CUDA	11.3（可选）
NumPy	1.19+
Pandas	1.2+
scikit-learn	0.24+
scipy	1.6+
tqdm	4.62+
matplotlib	3.4+
einops	0.4+

表 6.1 运行环境配置